数据库索引与查询优化

一、为什么需要索引

数据库表在磁盘上通常按主键顺序组织。如果没有索引，一次等值查询需要扫描整张表，复杂度是 O(N)。索引的本质是用额外的存储空间换取更少的磁盘读取次数。对于一门课程笔记来说，最重要的不是记住索引的名字，而是理解每种索引适合什么样的查询模式。

二、B+ 树索引

B+ 树是一种多路平衡查找树。非叶子节点只保存键值和指针，叶子节点保存完整记录或记录地址，并通过双向链表相连。这种结构带来三个直接好处：第一，树高通常只有三到四层，一次查询只需要少量随机 I/O；第二，叶子节点有序且相连，范围查询可以顺序扫描；第三，所有查询都落在叶子层，代价稳定。

在 MySQL 的 InnoDB 引擎中，主键索引就是聚簇索引，叶子节点直接存放整行数据；二级索引的叶子节点存放主键值，因此通过二级索引查询非索引列时需要进行一次回表。覆盖索引可以避免回表：如果查询需要的列都包含在二级索引中，优化器会直接使用索引返回结果。

最左前缀原则来自联合索引的排序方式。联合索引 (a, b, c) 等价于先按 a 排序，a 相同再按 b 排序，b 相同再按 c 排序。因此 WHERE b = 1 无法使用该索引，而 WHERE a = 1 AND c = 2 只能用到 a 这一列。

三、哈希索引

哈希索引用哈希函数把键映射到桶，等值查询的平均复杂度是 O(1)。它不支持范围查询，也不支持排序，因为哈希值本身没有顺序。哈希索引适合用户 ID、订单号这类只做精确匹配的场景。InnoDB 的自适应哈希索引由引擎自动维护，不需要用户显式创建。

四、执行计划与优化方法

EXPLAIN 是判断索引是否生效的第一工具。重点关注 type 列：const 表示主键或唯一索引等值查询，ref 表示普通索引等值查询，range 表示索引范围扫描，index 表示全索引扫描，ALL 表示全表扫描。key 列显示实际使用的索引，rows 列是优化器估算的扫描行数。

常见优化手段包括：为高频过滤条件建立索引；把选择性高的列放在联合索引前面；避免在索引列上使用函数或隐式类型转换；用覆盖索引减少回表；对长文本使用前缀索引；定期分析表和更新统计信息，避免优化器选错执行计划。

五、从数据库索引到向量索引

传统索引服务于精确匹配和范围查询，向量索引服务于语义相似度。HNSW 用多层跳表结构组织向量，上层稀疏、下层稠密，查询时从上层快速接近目标区域，再在下层精细搜索。IVF 则先用聚类把向量划分到多个倒排桶，查询时只扫描距离最近的若干个桶。两者都是用近似换速度，需要在召回率和延迟之间取舍。

在实际系统里，全文索引和向量索引经常同时使用：全文索引保证术语精确命中，向量索引保证语义召回，最后用 RRF 等融合算法合并两路结果。这正是混合检索的基本思路。